

P R O Y E C T O F I N A L M E T O D O S
N U M E R I C O S

Manual Técnico

Arquitectura · Algoritmos · Implementación · Pruebas

Python 3.11 · PyQt6 · NumPy · SymPy · Matplotlib · fpdf2

1. Arquitectura del Sistema

1.1 Diagrama General — Separación de Responsabilidades

El sistema implementa una arquitectura de tres capas con separación estricta de responsabilidades (SoC). La UI es 'tonta' (solo muestra datos), la lógica es 'ciega' (no sabe cómo se muestra). Esto permite cambiar la interfaz o los algoritmos de forma independiente.

CAPA UI (ui/) — PyQt6 · MathCanvas · Atomic Design
CAPA CORE (core/) — Python puro · NumPy · SymPy (sin PyQt6)
CAPA INFRASTRUCTURE (infrastructure/) — PDF · Gráficas · Historial JSON

1.2 Descripción de Componentes

Paquete / Módulo	Responsabilidad
core/raices/raices.py	Algoritmos de bisección, Newton-Raphson y Secante. Solo Python + NumPy + SymPy.
core/interpolacion/interpolacion.py	Interpolación de Lagrange y Newton con diferencias divididas.
core/sistemas/sistemas.py	Gauss-Seidel iterativo y Factorización LU.
core/integracion_derivacion/integracion_derivacion.py	Punto Medio, Trapecio, Simpson 1/3, Simpson 3/8 y Diferencias Finitas.
core/edos/edos.py	Método de Euler y Runge-Kutta de 4to orden (RK4).
ui/main_window.py	Ventana principal. Menú lateral, cambio de tema, enrutamiento de vistas.
ui/views/base_method_view.py	Clase abstracta base. Define estructura estándar: form + botones + tabs de resultados.
ui/views/*_views.py	Una vista por módulo. Hereda BaseMethodView e implementa _build_form y _run_calculation.
ui/components/math_canvas.py	Renderizador algebraico personalizado: muestra ecuaciones en notación visual.
ui/components/math_ast.py	Árbol sintáctico para parsear y construir expresiones matemáticas visuales.
ui/styles/	Tokens de diseño: colores semánticos, tipografía, espaciado, radios, temas claro/oscuro.
infrastructure/pdf_generator.py	Wrapper sobre fpdf2. Genera reportes PDF con tabla, procedimiento y gráfica.

infrastructure/plot_widget.py	Puente entre Matplotlib y PyQt6. Embebe la figura en la ventana nativa.
infrastructure/history_repo.py	CRUD sobre historial.json. Persiste cada cálculo con timestamp, módulo y parámetros.
main.py	Punto de entrada. Crea QApplication con estilo Fusion y fuente Segoe UI 10pt.

1.3 Flujo de Ejecución

1. main.py crea la QApplication y lanza MainWindow en pantalla completa.
2. El usuario elige un módulo desde el menú lateral; MainWindow carga la vista correspondiente en el panel principal.
3. La vista hereda BaseMethodView. Construye el formulario con `_build_form()`.
4. Al presionar 'Calcular', BaseMethodView llama a `_run_calculation()` (implementado en cada vista hija).
5. La vista hija llama a la función correspondiente del core (ej: `lagrange(x_points, y_points, x_eval)`).
6. El core retorna un dataclass inmutable (InterpolationResult, RootResult, etc.) con tabla, pasos y datos de gráfica.
7. BaseMethodView distribuye el resultado: tabla en QTableWidgetItem, pasos en QTextEdit, gráfica en PlotWidget.
8. Se guarda un HistoryRecord en history_repo y se activa el botón 'Exportar PDF'.

2. Especificaciones Técnicas

2.1 Dependencias y Bibliotecas

Librería	Versión mínima / Propósito
Python	3.10+ (se recomienda 3.11 LTS)
PyQt6	6.6.0 — Framework GUI multiplataforma basado en Qt6
NumPy	1.26.0 — Operaciones matriciales, arrays eficientes, linspace, cálculo numérico
SymPy	1.12 — Cálculo simbólico, simplificación, lambdify, sympify, derivadas automáticas
Matplotlib	3.8.0 — Generación de gráficas 2D integradas en la ventana PyQt6
fpdf2	2.7.0 — Generación de reportes PDF (wrapper en infrastructure/pdf_generator.py)

Todas las dependencias se instalan desde el archivo requirements.txt con:

```
pip install -r requirements.txt
```

2.2 Estructura de Archivos del Proyecto

```
ProyectoFinalMN-main/
├── main.py                # Punto de entrada
├── requirements.txt       # Dependencias
├── DOCUMENTACION.md
├── GUIA_INSTALACION.md
├── data/
│   └── historial.json     # Historial persistente
├── core/                 # Lógica matemática pura
│   ├── raices/raices.py
│   ├── interpolacion/interpolacion.py
│   ├── sistemas/sistemas.py
│   ├── integracion_derivacion/integracion_derivacion.py
│   └── edos/edos.py
├── ui/                   # Interfaz gráfica
│   ├── main_window.py
│   ├── views/
│   │   ├── base_method_view.py
│   │   ├── raices_views.py
│   │   ├── interpolacion_views.py
│   │   ├── sistemas_views.py
│   │   ├── integracion_views.py
│   │   ├── edos_views.py
│   │   ├── history_view.py
│   │   └── welcome_view.py
│   ├── components/
│   │   ├── math_canvas.py
│   │   ├── math_ast.py
│   │   └── math_input.py
│   └── styles/           # Tokens visuales y temas
```

```
└─ infrastructure/  
    └─ pdf_generator.py  
    └─ plot_widget.py  
    └─ history_repo.py
```

3. Módulos Implementados — Algoritmos

3.1 Módulo de Raíces (core/raices/raices.py)

Firma de funciones

```
biseccion(func_str, a, b, tolerance=1e-6, max_iterations=100) -> RootResult
newton_raphson(func_str, deriv_str, x0, tolerance=1e-6, max_iterations=100) ->
RootResult
secante(func_str, x0, x1, tolerance=1e-6, max_iterations=100) -> RootResult
```

Bisección — Algoritmo

Punto medio del intervalo
$$c = (a + b) / 2$$

- Precondición: $f(a) \cdot f(b) < 0$. Si no se cumple, lanza `ValueError` con mensaje descriptivo.
- En cada iteración: calcula $c = (a+b)/2$, evalúa $f(c)$. Si $f(a) \cdot f(c) < 0$ el nuevo $b = c$, si no el nuevo $a = c$.
- Convergencia: $|b - a| < \text{tolerancia}$ o $f(c) = 0$ exactamente.
- Complejidad: $O(\log_2((b-a)/\text{tolerancia}))$ iteraciones.

Newton-Raphson — Algoritmo

Fórmula de iteración
$$x_{n+1} = x_n - f(x_n) / f'(x_n)$$

- La derivada $f'(x)$ se pasa como string separado y se parsea con SymPy (`sympify + lambdify`).
- Detención si $|f'(x_n)| < 1e-14$ para evitar división por cero.
- Error calculado como $|x_{n+1} - x_n|$.

Secante — Algoritmo

Fórmula de la Secante
$$x_{n+1} = x_n - f(x_n) * (x_n - x_{n-1}) / (f(x_n) - f(x_{n-1}))$$

- No requiere derivada analítica. Aproxima la pendiente con dos puntos anteriores.
- Requiere dos valores iniciales distintos x_0 y x_1 .

Dataclasses de resultado

Clase	Campos
BiseccionRow	iteration, a, c, b, fa, fc, fb, error
NewtonRaphsonRow	iteration, xn, fxn, fpxn, xn1, fxn1, error
SecanteRow	iteration, xn_prev, xn, fxn_prev, fxn, xn1, error
RootResult	root, iterations, error, converged, table, procedure_steps, message

3.2 Módulo de Interpolación (core/interpolacion/interpolacion.py)**Firma de funciones**

```
lagrange(x_points, y_points, x_eval=None) -> InterpolationResult
newton_interpolation(x_points, y_points, x_eval=None) -> InterpolationResult
```

Lagrange — Algoritmo

Polinomio base

$$L_i(x) = \prod_{j \neq i} (x - x_j) / (x_i - x_j)$$

Polinomio interpolante

$$P(x) = \sum_{i=0..n} y_i * L_i(x)$$

- Usa SymPy (Symbol, expand, simplify, lambdify) para construir el polinomio simbólico.
- format_poly() redondea coeficientes y reemplaza ** por ^ y * implícito para la vista.
- La curva de gráfica se genera con 200 puntos en [xmin-20%, xmax+20%] usando np.linspace.
- Complejidad: $O(n^2)$ para construir el polinomio; $O(n)$ por evaluación.

Newton Diferencias Divididas — Algoritmo

Tabla triangular (orden k)

$$f[x_i, \dots, x_{i+k}] = (f[x_{i+1}, \dots] - f[x_i, \dots]) / (x_{i+k} - x_i)$$

Polinomio de Newton

$$P(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \dots$$

- La tabla diff_table se construye como np.zeros((n, n)). Los coeficientes son diff_table[0, :].
- Ventaja: al agregar un nuevo punto solo se añade una columna a la tabla; coeficientes anteriores no cambian.

- Complejidad: $O(n^2)$ para la tabla, $O(n)$ para la evaluación.

Dataclass de resultado compartido

Clase	Campos
LagrangeRow	i, xi, yi, li_value, term, accumulated
NewtonInterpRow	i, xi, yi, divided_diff, coefficient, accumulated
InterpolationResult	polynomial_str, polynomial_math, interpolated_value, table, procedure_steps, x_plot, y_plot, message

3.3 Módulo de Sistemas (core/sistemas/sistemas.py)

Firma de funciones

```
gauss_seidel(matrix_a, vector_b, x0=None, tolerance=1e-6, max_iterations=100) -> SystemResult
lu_factorization(matrix_a, vector_b) -> SystemResult
```

Gauss-Seidel — Algoritmo

$$\begin{array}{c} \text{Actualización de } x_i \\ x_i^{(k+1)} = (b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)}) / a_{ii} \end{array}$$

- Verifica dominancia diagonal antes de iterar. Emite advertencia si no se cumple (convergencia no garantizada).
- Verifica que todos los elementos diagonales sean $\neq 0$ (lanza ValueError si hay cero).
- Error calculado como norma infinito: $\max|x_{\text{nuevo}} - x_{\text{viejo}}|$.

Factorización LU

- Descompone $A = L \cdot U$ usando eliminación gaussiana con pivoteo parcial.
- Resuelve $Ly = b$ por sustitución hacia adelante, luego $Ux = y$ por sustitución hacia atrás.
- No es iterativo: entrega resultado exacto (salvo errores de punto flotante).

3.4 Módulo de Integración y Derivación (core/integracion_derivacion/integracion_derivacion.py)

Firma de funciones

```
punto_medio(func_str, a, b, n_intervals) -> IntegrationResult
trapecio(func_str, a, b, n_intervals) -> IntegrationResult
simpson(func_str, a, b, n_intervals, variant='1/3') -> IntegrationResult
```


Fórmulas

Punto Medio

$$I = \text{sum}[f(x_{\text{mid}_i}) * dx] \quad \text{donde } x_{\text{mid}_i} = (x_i + x_{i+1}) / 2$$

Trapecio compuesto

$$I = (dx/2) * [f(x_0) + 2*\text{sum}(f(x_i)) + f(x_n)]$$

Simpson 1/3 compuesto

$$I = (dx/3) * [f(x_0) + 4*f(x_1) + 2*f(x_2) + \dots + f(x_n)] \quad (n \text{ par})$$

Diferencias centradas (1er orden)

$$f'(x) \approx [f(x+h) - f(x-h)] / (2h)$$

Derivación por Diferencias Finitas

- Diferencias hacia adelante: $f'(x) \approx [f(x+h) - f(x)] / h$ — error $O(h)$.
- Diferencias hacia atrás: $f'(x) \approx [f(x) - f(x-h)] / h$ — error $O(h)$.
- Diferencias centradas: $f'(x) \approx [f(x+h) - f(x-h)] / (2h)$ — error $O(h^2)$, mayor precisión.

3.5 Módulo de EDOs (core/edos/edos.py)

Firma de funciones

```
euler(func_str, x0, y0, x_final, h) -> ODEResult
runge_kutta(func_str, x0, y0, x_final, h) -> ODEResult
```

Euler — Algoritmo

Fórmula de Euler

$$y_{i+1} = y_i + h * f(x_i, y_i)$$

- $n_steps = \text{ceil}((x_final - x_0) / h)$. Valida que $h > 0$ y $x_final > x_0$.
- Error estimado comparando con el paso anterior cuando se dispone de dos valores consecutivos.

Runge-Kutta 4 (RK4) — Algoritmo

Pendientes k1..k4

```
k1=h*f(xi,yi)  k2=h*f(xi+h/2, yi+k1/2)  k3=h*f(xi+h/2, yi+k2/2)
                k4=h*f(xi+h, yi+k3)
```

Actualización

$$y_{i+1} = y_i + (k1 + 2k2 + 2k3 + k4) / 6$$

- Cuatro evaluaciones de f por paso. Error local $O(h^5)$, error global $O(h^4)$.
- Significativamente más preciso que Euler para el mismo tamaño de paso h .

4. Capa de Interfaz de Usuario (ui/)

4.1 Patrón BaseMethodView

Todas las vistas de métodos heredan de BaseMethodView (ui/views/base_method_view.py). Esta clase abstracta define la estructura visual estándar y concentra la lógica repetida: historial, PDF, gráficas.

Métodos que cada vista hija **DEBE** implementar:

Método abstracto	Responsabilidad
_build_form()	Retorna un QWidget con los campos de entrada del método.
_get_method_name()	Retorna el nombre legible del método (para cabecera y PDF).
_get_module_name()	Retorna el nombre del módulo ('Raíces', 'Interpolación', etc.).
_run_calculation()	Llama al core, obtiene el resultado y llama a los métodos de display.
_load_example()	Carga datos de ejemplo en los campos del formulario.
_get_parameters()	Retorna dict con los parámetros actuales para el historial.

Responsabilidades que BaseMethodView provee automáticamente:

- Estructura visual: cabecera, formulario, botones 'Calcular', 'Ejemplo' y 'Exportar PDF'.
- Distribución de resultados: _display_table(), _display_steps(), _display_graph().
- Guardado automático en HistoryRepository tras cada cálculo exitoso.
- Generación del PDF via PdfGenerator con todos los datos del resultado.

4.2 Componentes Reutilizables

Componente	Descripción
MathCanvas	Renderizador algebraico personalizado. Muestra ecuaciones en notación visual (x^2 , fracciones, etc.) en tiempo real.
MathExpression / build_ast_from_text	Árbol sintáctico (AST) que parsea texto matemático y genera la representación visual para MathCanvas.
PlotWidget	Embebe una figura de Matplotlib dentro de un QWidget nativo de PyQt6. Permite zoom e inspección.
ModuleCard (welcome_view)	Tarjeta interactiva con ícono, nombre y descripción del módulo. Emite señal clicked(str) al hacer clic.

4.3 Sistema de Tokens Visuales (ui/styles/)

El sistema de diseño sigue la regla de no usar 'magic numbers'. Todos los valores visuales se definen como constantes semánticas:

- Colors: colores semánticos (primary, danger, success, background, surface, text).
- Spacing: XS, SMALL, MEDIUM, LARGE, XL — en píxeles.
- Typography: tamaños de fuente (CAPTION, BODY, SUBTITLE, TITLE, DISPLAY).
- Radius: radios de borde para componentes redondeados.
- ThemeManager: maneja el cambio entre tema claro y oscuro en toda la aplicación.

5. Capa de Infraestructura (infrastructure/)

5.1 PdfGenerator (pdf_generator.py)

Wrapper sobre la librería fpdf2. Si se desea cambiar de fpdf2 a otra librería, solo se modifica este archivo.

Método principal:

```
generate_report(method_name, parameters, procedure_text, table_headers,
                table_rows, result_summary, observations, chart_image_path,
                output_path) -> str
```

- `_sanitize(text)`: reemplaza caracteres Unicode no soportados por fpdf2 ($\approx$, π , $\int$, Δ , $\sqrt{}$, etc.) por equivalentes ASCII.
- La gráfica se guarda como imagen temporal (tempfile) y se incrusta en el PDF.
- Retorna la ruta del archivo PDF generado.

5.2 PlotWidget (plot_widget.py)

- Hereda de `QWidget`. Embebe un `FigureCanvasQTAgg` de Matplotlib dentro de PyQt6.
- Recibe listas `x_plot` e `y_plot` del resultado del core y dibuja la curva con los puntos de datos.
- Soporte de zoom interactivo mediante la barra de herramientas de Matplotlib.

5.3 HistoryRepository (history_repo.py)

- Persiste los cálculos en `data/historial.json` (archivo JSON, no SQLite directamente).
- Interfaz CRUD: `load_all()`, `save(record)`, `delete(index)`, `clear_all()`.
- Cada registro es un `HistoryRecord` (dataclass) con: `timestamp`, `module`, `method`, `parameters`, `result_summary`, `full_data`.
- Los registros más recientes se insertan al inicio de la lista (`insert(0, record)`).

6. Pruebas y Validación

6.1 Estrategia de Pruebas

Cada función del core puede probarse de forma completamente independiente de la UI, ya que no hay importaciones de PyQt6 en los módulos core/. Esto facilita las pruebas unitarias.

Tipo de prueba	Descripción
Prueba unitaria de cálculo	Llamar directamente a <code>biseccion()</code> , <code>lagrange()</code> , etc. con entradas conocidas y verificar el resultado.
Prueba de validación de entrada	Verificar que <code>ValueError</code> se lanza con mensajes correctos para entradas inválidas.
Prueba de convergencia	Verificar que <code>result.converged == True</code> y <code>result.error < tolerancia</code> para casos conocidos.
Prueba del dataclass resultado	Verificar que <code>result.table</code> contiene el número correcto de filas y tipos correctos.
Prueba de integración UI	Verificar que <code>BaseMethodView</code> llama correctamente a <code>_run_calculation()</code> sin excepción.

6.2 Casos de Prueba Representativos

Función	Entrada y resultado esperado
<code>biseccion</code>	$f='x^3-x-2'$, $a=1$, $b=2 \rightarrow \text{root} \approx 1.3247180$, <code>converged=True</code>
<code>lagrange</code>	$x=[0,1,2]$, $y=[0,1,4]$, $x_eval=1.5 \rightarrow P(1.5) = 2.25$ (exacto para x^2)
<code>newton_interpolation</code>	$x=[1,2,3,4]$, $y=[1,4,9,16]$, $x_eval=2.5 \rightarrow P(2.5) = 6.25$ (exacto para x^2)
<code>gauss_seidel</code>	$2x+y=5$, $x+3y=10 \rightarrow x=1.0$, $y=3.0$, <code>converged=True</code>
<code>trapecio</code>	$f='x^2'$, $a=0$, $b=1$, $n=100 \rightarrow I \approx 0.3333$ (valor exacto: $1/3$)
<code>euler</code>	$dy/dx=y$, $x_0=0$, $y_0=1$, $x_final=1$, $h=0.1 \rightarrow y(1) \approx 2.5937$ (exacto: $e \approx 2.71828$)
<code>runge_kutta</code>	$dy/dx=y$, $x_0=0$, $y_0=1$, $x_final=1$, $h=0.5 \rightarrow y(1) \approx 2.71828$ (muy cercano a e)
<code>biseccion (error)</code>	$f='x^2+1'$, $a=-1$, $b=1 \rightarrow \text{ValueError: 'No hay cambio de signo'}$
<code>lagrange (error)</code>	$x=[1,1,2]$, $y=[0,1,2] \rightarrow \text{ValueError: 'Los valores de x no deben repetirse'}$

6.3 Limitaciones Conocidas

- Fenómeno de Runge: para interpolación con más de ~10 puntos equidistantes, el polinomio puede oscilar fuertemente en los extremos. Considerar nodos de Chebyshev.
- Newton-Raphson: si $f'(x_0) \approx 0$ en el punto inicial, el método puede no converger. Se detecta y lanza error.
- Gauss-Seidel: sin dominancia diagonal, la convergencia no está garantizada. La aplicación lo advierte.
- Simpson 1/3 requiere n par. La vista valida esta condición antes de llamar al core.
- El historial no tiene límite de registros. Para archivos historial.json muy grandes, la lectura puede ralentizarse.
- La derivada para Newton-Raphson debe ingresarse manualmente; no se calcula automáticamente en la vista.

7. Mantenimiento y Extensión

7.1 Cómo Agregar un Nuevo Método Numérico

9. Crear la función en el módulo core/ correspondiente (o crear un nuevo archivo si es un módulo nuevo).
10. Definir un dataclass de resultado (frozen=True) con todos los datos necesarios: table, procedure_steps, message.
11. Crear una nueva vista en ui/views/ heredando de BaseMethodView e implementar los 6 métodos requeridos.
12. Registrar la nueva vista en ui/main_window.py: agregar la entrada al menú lateral y el enrutamiento.
13. Opcional: agregar tarjeta en welcome_view.py si es un módulo completamente nuevo.

Regla de oro

La función del core NUNCA debe importar PyQt6, matplotlib ni ninguna librería de infraestructura. Solo Python estándar + NumPy + SymPy. Esto garantiza que pueda probarse y reutilizarse de forma independiente.

7.2 Mejoras Futuras Propuestas

Mejora	Descripción técnica
Derivada automática en Newton-Raphson	Usar <code>sympy.diff()</code> para calcular $f'(x)$ automáticamente desde $f(x)$ ingresada como string.
Nodos de Chebyshev en interpolación	Agregar opción para generar automáticamente x_i como nodos de Chebyshev y reducir el error de Runge.
Ajuste de curvas (mínimos cuadrados)	Nuevo módulo core/ajuste/ con regresión lineal y polinomial usando <code>np.polyfit</code> .
Análisis de error	Mostrar cota de error de truncamiento para cada método (disponible analíticamente para Simpson y Trapecio).
Exportación de historial a CSV	Agregar método <code>HistoryRepository.export_csv()</code> para análisis externo.
Modo presentación	Botón para agrandar la gráfica a pantalla completa, útil para presentaciones académicas.